Need for High-Speed Computing:

Computing: Using a computer to process data and produce results.

Examples:

- Calculating marks of students
- Predicting tomorrow's weather
- Detecting disease from medical images
- Simulating traffic, climate

As problems grow larger and more complex, computing needs to be

- Faster
- More accurate
- More scalable

This leads us to high-speed computing.

High-speed computing: The ability of a computing system to perform a very large number of computations per second to solve time-critical and data-intensive problems.

Speed is usually measured in:

- **FLOPS** – Floating Point Operations Per Second
    - MFLOPS (10^6), GFLOPS (10^9)

Why do we need high-speed computing?

Example 1: Weather Forecasting

- Weather data comes from:
    - Satellites, Radars, Sensors
- Millions of data points
- Complex mathematical equations

If processed slowly:

- Forecast arrives after the weather event

High-speed computing enables:

- Real-time prediction
- Disaster warnings (cyclones, floods)

Example 2: Medical & Healthcare

- Genome sequencing
- Drug discovery simulations

A single DNA sequence:

- Billions of characters

Without high-speed computing:

- Analysis may take years

Example 3: Finance & Business

- Stock market analysis

Delays of milliseconds can mean:

- Huge financial losses

Limitations of a single (sequential) computer:

Limitation 1: Clock Speed Saturation

Earlier:

- Faster computers = higher clock speed (GHz)

Now:

- Heat generation
- Power consumption
- Hardware reliability issues

Clock speed cannot increase indefinitely.

Limitation 2: Sequential Execution

Traditional programs:

- One instruction at a time
- One CPU core

Limitation 3: Memory & Data Bottleneck

- Large data needs:
    - Huge memory
    - Fast access
- Single system struggles with:
    - Bandwidth
    - Latency

Parallel computing: Instead of solving a problem sequentially, divide it into multiple smaller parts and solve them simultaneously.

Analogy:

Cleaning a House

- 1 person cleaning 10 rooms → very slow
- 10 people cleaning 1 room each → very fast

Same work, Less time

This is the core idea of parallel computing.

Why High-Speed Computing naturally leads to parallel computing?

Problem	Sequential Computer	Parallel Computer
Huge data	Slow	Fast
Time-critical	Fails	Succeeds
Complex models	Impossible	Possible
Scalability	Limited	High

Can parallel programs run on non-parallel hardware?
No

Parallel computer: is a computing system that contains multiple processing units which work simultaneously to solve a problem by dividing it into smaller sub-problems.

Why do we need parallel computers?

Reason 1: Physical Limits of Single-Core CPUs

Earlier improvement:

- Increase clock frequency (MHz → GHz)

Problem:

- Excessive heat
- Power inefficiency
- Hardware damage

Reason 2: Instruction-Level Parallelism has limits

Techniques like:

- Pipelining
- Superscalar execution

- Branch prediction

Eventually face:

- Data dependencies
- Control hazards

Reason 3: Memory Speed is Slower than CPU Speed

- CPU speed grows rapidly
- Memory speed grows slowly

Result:

- CPU waits idle for data

Evolution: From Single-Core to Multi-Core

Old Computers: One CPU, One core, One instruction stream

Modern Computers: Multiple cores on one chip, Shared or private caches, Parallel execution

Example: Dual-core, Quad-core, Octa-core CPUs

Even mobile phone is parallel today.

Types of Parallel Computers:

Multi-Core Systems

- Multiple cores on one CPU chip
- Shared memory
- Used in desktops & laptops

Multiprocessor Systems

- Multiple CPUs
- Shared memory
- High-end servers

Clusters

- Many computers connected by network
- Each has its own memory
- Used in research labs

Supercomputers

- Thousands to millions of cores
- Extremely high performance
- Used for national-level problems

Why parallel computers are better than fast computers with single CPU?

Aspect	Fast Single CPU	Parallel Computer
Clock speed	Limited	Moderate
Scalability	Poor	Excellent
Energy efficiency	Low	High
Cost effectiveness	Low	High
Fault tolerance	Low	High

Supercomputer: It is a high-performance computing system capable of executing billions to quintillions of calculations per second, primarily used for scientific, engineering, and national-scale problems.

Key idea:

- Supercomputers are always parallel computers.
- Performance is measured in FLOPS.

Generations of supercomputers:

Phase 1: Early Supercomputers (1960s–1970s)

Example: CDC 6600 (1964)

Key Characteristics:

- Single powerful CPU
- Very high clock speed (for its time)
- Focus on hardware optimization

Limitation:

- Still sequential
- Speed improvement depended on faster hardware

This approach quickly hit physical limits.

Phase 2: Vector Supercomputers (1970s–1980s)

Core Idea: Vector Processing: Performs the same operation on many data elements at once

Example:

```
for i = 1 to 1000
    A[i] = B[i] + C[i]
```

Instead of 1000 separate instructions, one vector instruction is needed.

Example: Cray-1

Advantages:

- Very fast for scientific calculations
- Excellent for matrix operations

Limitations:

- Expensive
- Limited flexibility
- Not suitable for all problems

Phase 3: Massively Parallel Processing (MPP) Systems (1990s)

Core Idea: Uses hundreds or thousands of processors working together

Architecture:

- Each processor has its own memory
- Communication via network

Advantages:

- High scalability
- Better fault tolerance

Limitations:

- Complex programming
- Communication overhead

This phase marked the true beginning of parallel supercomputing**.**

Phase 4: Cluster-Based Supercomputers (2000s)

Key Idea: Connects commodity computers using high-speed networks

Characteristics:

- Low cost compared to vector machines
- Uses standard CPUs
- Highly scalable
- MPI for communication

Most modern supercomputers are clusters.

Phase 5: Hybrid Supercomputers (2010s–Present)

Core Idea: Combines different types of processors

Example: CPU + GPU, CPU + accelerator

Why GPUs?

- Thousands of lightweight cores
- Excellent for data-parallel workloads

⟐ Advantages

- Massive performance boost
- Energy efficiency

Almost all top supercomputers today are hybrid systems**.**

Why Parallelism Dominates Supercomputing?

- Clock speed scaling ended
- Energy efficiency matters
- Data sizes exploded
- Parallel architectures scale better

Modern parallel computer: These are computing systems that use multiple processing elements simultaneously, often combining multi-core CPUs, accelerators (GPUs), and distributed systems, to achieve high performance and scalability.

Key idea: Parallelism exists at multiple levels**.**

Levels of Parallelism in Modern Systems:

Level	Description
Instruction-level	Pipelining, SIMD
Core-level	Multiple cores
Node-level	Multiple CPUs
System-level	Clusters of nodes

Multi-Core CPU: A single chip containing multiple independent processing cores.

Example: Dual-core, Quad-core, Octa-core

Each core:

- Can execute its own program
- Shares or has private cache

Why Multi-Core CPUs?

- Energy efficient
- Better performance per watt
- Exploit thread-level parallelism

Shared Memory Parallel Computers:

Concept:

- All processors access the same memory
- Communication via shared variables

Models:

- UMA (Uniform Memory Access): Equal access time for all processors
- NUMA (Non-Uniform Memory Access): Access time depends on memory location

Advantages:

- Easy programming
- Fast communication

Disadvantages:

- Limited scalability
- Memory contention

GPUs as parallel computers: Originally for graphics. Now used for Machine learning, Scientific simulations, Image processing.

Key Feature:

- Thousands of small cores
- Best for data-parallel tasks

CPU vs GPU:

Feature	CPU	GPU
Cores	Few (4–64)	Thousands
Control	Complex	Simple
Best for	Control tasks	Data-parallel tasks

Distributed Memory Systems (Clusters):

Cluster: A group of independent computers connected via a network. Each node Has its own memory, communicates using messages.

Characteristics:

- Highly scalable
- Fault tolerant
- Programming is complex

Used in: Universities, Research labs, Cloud data centers

Hybrid Parallel Computers:

Modern HPC systems combine multi-core CPUs, GPUs, cluster architecture.

Example: CPU + GPU nodes connected in a cluster

This is the dominant architecture today.

Shared vs Distributed Memory:

Aspect	Shared Memory	Distributed Memory
Memory	Common	Separate
Communication	Implicit	Explicit (messages)
Scalability	Limited	High
Programming	Easier	Harder

Need for writing parallel program:

If modern computers are already parallel, why can't they automatically run programs faster?

Because most programs are written sequentially. Parallel hardware does not automatically make a program parallel.

Sequential program vs parallel program:

Sequential program:

- One flow of control
- One instruction after another
- Designed for single-core CPUs

Parallel program:

- Multiple flows of control
- Multiple tasks executed simultaneously
- Explicitly exposes parallelism

Concurrency vs Parallelism:

Concurrency:

- Multiple tasks in progress
- May or may not run simultaneously

Parallelism:

- Multiple tasks running at the same time

Parallelism requires parallel hardware and parallel software.

Why parallel programming is hard:

Parallel programs introduce

- Race conditions
- Deadlocks
- Load imbalance
- Debugging difficulty

Why is parallel programming difficult?

Because multiple tasks execute at the same time**,** and they may interact in unpredictable ways**.**

In sequential programming:

- Order is fixed
- Behavior is predictable

In parallel programming:

- Order may change
- Results may vary if not handled carefully

Major challenges in parallel programming:

Challenge 1: Task Decomposition: Breaking a large problem into smaller parallel tasks.

Problem:

- Too few tasks lead to poor parallelism
- Too many tasks lead to overhead

Example: Matrix multiplication: Divide by rows? / Columns? / Blocks?

Choosing the wrong method affects performance.

Challenge 2: Data Dependency: When one task depends on the result of another task.

Types:

- True dependency
- Anti-dependency
- Output dependency

Example:

A = B + C
D = A + E

Second operation must wait.

Challenge 3: Synchronization: Coordinating access to shared resources.

Problems:

- Race conditions
- Incorrect results

Example:

Two threads updating the same variable:

x = x + 1

Without synchronization:

- Final value is unpredictable

Challenge 4: Communication Overhead: Time spent exchanging data between processors.

In shared memory:

- Cache coherence
- Memory contention

In distributed memory:

- Network latency
- Bandwidth limitations

Too much communication kills performance.

Challenge 5: Load Imbalance: Unequal work distribution among processors.

Example:

- Some processors finish early
- Others keep working

Result:

- Idle processors
- Wasted resources

Challenge 6: Scalability: Ability to maintain performance as processors increase.

Issues:

- Communication increases
- Synchronization increases
- Serial bottlenecks dominate

Challenge 7: Debugging & Testing:

Parallel bugs are:

- Non-deterministic
- Hard to reproduce

Examples:

- Deadlocks
- Race conditions

Data parallelism: It is a form of parallelism where the same operation is performed simultaneously on different pieces of data using multiple processing elements.

Key idea:

- Same instruction
- Different data
- Executed in parallel

Example:

Consider adding two arrays:

A[i] = B[i] + C[i]

Each index operation:

- Independent
- Can be done in parallel

So instead of one processor doing all additions, we use many processors, each handling part of the data.

Characteristics of data parallelism:

Feature	Description
Parallel unit	Data elements
Operation	Same
Control flow	Identical
Synchronization	Minimal
Granularity	Fine-grained

Where is data parallelism used?

Scientific computing:

- Matrix operations
- Numerical simulations

Image & video processing:

- Pixel-wise operations
- Filters

Machine learning:

- Vector and matrix computations
- Neural networks

GPUs:

- Designed primarily for data parallel workloads

Data Parallelism vs SIMD:

SIMD = Single Instruction, Multiple Data

- One instruction
- Applied to many data items

SIMD is a hardware form of data parallelism.

Advantages of data parallelism:

- Easy to implement
- High scalability
- Minimal synchronization
- Excellent performance on GPUs

Limitations of data parallelism:

- Not suitable for dependent data
- Control flow divergence reduces efficiency
- Less flexible than task parallelism

Task parallelism: It is a form of parallelism where different tasks or functions are executed simultaneously on multiple processors or cores.

Key idea:

- Different tasks

- Possibly different operations
- May work on same or different data

Task parallelism in computing:

- A web server handles client request, database access, logging, file transfer. Each request or service is a separate task which can be handled in parallel.
- A simulation program preprocesses data, does numerical computation, perform visualization, stores result. Each task has different code and can run concurrently.

Characteristics of task parallelism:

Feature	Description
Parallel unit	Task / Function
Operations	Different
Control flow	Independent
Data sharing	Possible
Granularity	Coarse-grained

Advantages of task parallelism:

- Exploits functional independence
- Suitable for complex applications
- Improves system throughput
- Natural for real-world workflows

Limitations of task parallelism:

- Harder to balance load
- Tasks may depend on each other
- Limited parallelism if tasks are few
- Synchronization complexity

When to use task parallelism?

Use it when:

- Tasks are independent
- Tasks perform different operations
- Workflow-style applications
- Server-side and system software

Comparison of data parallelism and task parallelism:

Aspect	Data Parallelism	Task Parallelism
Basic unit	Data	Task / Function
Operation	Same	Different
Control flow	Identical	Different
Granularity	Fine-grained	Coarse-grained
Synchronization	Low	Higher
Scalability	Very high	Limited
Load balancing	Easier	Harder
Hardware suitability	GPU, SIMD	CPU, Multicore
Programming complexity	Lower	Higher

Real world usage:

Application	Parallelism Type
Matrix computation	Data
Image processing	Data
Deep learning	Data
Web servers	Task
Operating systems	Task
Scientific pipelines	Both